

Полное руководство по конфигурированию UI через ui_config.json

Оглавление

- 1. Введение и ключевая концепция
 - Ключевая концепция: Data-First
- 2. Структура файла ui_config.json
- 3. Конфигурация отдельной схемы (например, "brief")
 - 3.1. Группировка полей (field_groups)
- 4. Конфигурация полей (field_config) - Детальный разбор
 - 4.1. Поведение по умолчанию
 - 4.2. Общие параметры для всех виджетов
 - 4.3. Скрытие полей: два способа
- 5. universal_array_editor — редактор массивов объектов
 - 5.1. Интерфейсы (interface)
 - 5.2. Основные параметры
 - 5.3. Параметры для интерфейса tabs
 - 5.4. Параметры для интерфейса dropdown_selector
 - 5.5. Конфигурация полей внутри элемента массива
 - 5.6. Вложенная группировка полей
- 6. Дополнительные нюансы, поведение и ограничения
- 7. Примеры для каждого нюанса
 - 7.1. dropdown_selector: display_field / display_fields / display_separator / hide_*
 - 7.2. Верхнеуровневые массивы (items)
 - 7.3. Когда применяется конфиг items к объекту
 - 7.4. inline: группировка в одну строку
 - 7.5. Ограничения nested_group
 - 7.6. Где работает field_order
 - 7.7. placeholder как дефолт при создании
 - 7.8. combobox: readonly и добавление текущего значения
 - 7.9. list_editor: фильтрация пустых строк
 - 7.10. Приоритет меток

- [7.11. Валидация required](#)
- [7.12. collapsed: когда работает](#)
- [7.13. interface vs widget](#)

1. Введение и ключевая концепция

Этот документ — исчерпывающее руководство по настройке пользовательского интерфейса в StoryBook Manager с помощью файла `config/ui_config.json`.

Система построения интерфейса основана на классе `UniversalFormGenerator` (`gui/universal_json_editor.py`), который динамически создает формы для редактирования JSON-файлов.

Ключевая концепция: "Data-First" (Сначала данные)

Важно понимать фундаментальный принцип работы редактора:

1. **Сначала он смотрит на структуру ваших данных** (загруженного JSON-файла).
2. На основе этой структуры он строит базовый интерфейс, показывая все поля с виджетами по умолчанию.
3. И только потом он **применяет настройки из `ui_config.json`**, чтобы "декорировать" и кастомизировать уже созданный интерфейс.

Это объясняет, почему вы видите все поля из вашего JSON, даже если они не описаны в `ui_config.json`.

Ключевые возможности:

- **Быстрая кастомизация:** изменяйте типы виджетов, их размеры, метки и порядок без единой строчки кода.
- **Логическая группировка:** объединяйте связанные поля в группы, аккордеоны или располагайте их в несколько колонок.
- **Работа со сложными данными:** легко редактируйте вложенные объекты и массивы объектов с помощью мощного виджета `universal_array_editor`.
- **Контекстная настройка:** задавайте разные правила отображения для разных типов файлов (`brief`, `shots`, `screenplay` и т.д.).
- **Скрытие полей:** скрывайте ненужные поля из интерфейса.
- **Поведение по умолчанию:** если поле не описано в `field_config`, `UniversalFormGenerator` попытается "угадать" подходящий виджет на основе типа

данных этого поля в JSON.

2. Структура файла `ui_config.json`

Файл имеет четкую иерархическую структуру. На верхнем уровне находятся три ключевых объекта:

1. `"translations"` : Глобальный словарь для перевода имен полей (ключей JSON) в человекочитаемые метки.
2. `"editor_settings"` : Общие настройки редактора.
3. `"schema_configs"` : **Основной раздел**, где хранятся конфигурации для каждого типа JSON-файла.

```
{
  "translations": {
    "title": "Название",
    "description": "Описание"
    // ... другие переводы
  },
  "editor_settings": {
    "use_universal_editor": true, // Глобально вкл/выкл универсальный редактор
    "fallback_to_legacy": true,
    "debug_mode": false
  },
  "brief": {
    // ... конфигурация для файлов типа "brief"
  },
  "shots": {
    // ... конфигурация для файлов типа "shots"
  },
  "screenplay": {
    // ... конфигурация для файлов типа "screenplay"
  }
  // ... другие схемы
}
```

Важно: Имена ключей в корневом объекте (`brief` , `shots` , `screenplay`) должны точно совпадать с типом редактируемого файла.

3. Конфигурация отдельной схемы (например, "brief")

Каждый объект конфигурации схемы (например, "brief": {}) содержит два главных раздела:

1. "field_groups" : Определяет, как поля группируются и располагаются на форме.
2. "field_config" : Определяет, как выглядит и ведет себя *каждое отдельное поле*.

```
"brief": {  
  "field_groups": {  
    "basic_info": {  
      "title": "Основная информация",  
      "fields": ["title", "genre", "target_age", "language"],  
      "layout": "grid",  
      "columns": 2  
    }  
    // ... другие группы  
  },  
  "field_config": {  
    "title": {  
      "widget": "entry",  
      "width": 50  
    },  
    "description": {  
      "widget": "text_area",  
      "height": 4  
    }  
    // ... другие поля  
  }  
}
```

3.1. Группировка полей (field_groups)

Этот раздел отвечает за визуальную компоновку. Он представляет собой объект, где каждый ключ — это уникальное имя группы.

Параметры группы:

Параметр	Тип	Описание	Пример
title	string	Заголовок, который будет отображаться над группой.	"Основная информация"
fields	array	Массив строк с именами полей (ключей из JSON), которые должны войти в эту группу.	["title", "genre", "target_age"]
layout	string	Тип компоновки: "vertical" (по умолчанию) или "grid" .	"grid"
columns	integer	Только для layout: "grid" . Количество колонок в сетке.	2
collapsible	boolean	Если true , группа превращается в сворачиваемый аккордеон.	true
collapsed	boolean	Только для collapsible: true . Если true , аккордеон будет свернут по умолчанию.	true
hide_label	boolean	Если true , заголовок (title) группы не будет отображаться. Полезно для создания "невидимых" контейнеров.	true

Пример аккордеона:

```
"additional": {  
  "title": "Дополнительно",  
  "fields": ["moral"],  
  "collapsible": true,  
  "collapsed": true  
}
```

Пример использования Wildcard * :

Иногда нужно создать группу для "всех остальных" полей. Вместо того чтобы перечислять их, можно использовать * .

```
"additional_fields": {  
  "title": "Дополнительные настройки",  
  "fields": ["*"], // Включает все поля, не упомянутые в других группах  
  "collapsible": true,  
  "collapsed": true  
}
```

4. Конфигурация полей (field_config) - Детальный разбор

Это "сердце" всей системы. Здесь вы описываете каждый виджет. Ключи в field_config — это имена полей из вашего JSON-файла.

4.1. Поведение по умолчанию (когда конфигурации нет)

Это важно понимать: вам не нужно описывать *каждое* поле. Если поле из вашего JSON-файла отсутствует в field_config , UniversalFormGenerator не сломается. Вместо этого он попытается "угадать" подходящий виджет на основе типа данных этого поля в JSON.

Правила по умолчанию:

Тип данных в JSON	Виджет по умолчанию
string	entry (однострочное поле)
integer , number	entry

Тип данных в JSON	Виджет по умолчанию
boolean	checkbox
array (массив строк)	list_editor
array (массив объектов)	universal_array_editor с interface: "dropdown_selector"
object	nested_group

Это позволяет быстро начать работу: сначала редактор покажет все поля с виджетами по умолчанию, а вы сможете постепенно добавлять конфигурации в `ui_config.json` только для тех полей, которые требуют кастомизации.

4.2. Общие параметры для всех виджетов

Параметр	Тип	Описание
widget	string	Обязательный параметр. Тип визуального элемента (см. ниже).
label	string	Явно задает метку для поля, переопределяя <code>translations</code> .
hide_label	boolean	Если <code>true</code> , метка поля не будет показана.
readonly	boolean	Если <code>true</code> , поле будет доступно только для чтения.
hidden	boolean	Если <code>true</code> , поле будет полностью скрыто из интерфейса.
inline	boolean	Если <code>true</code> , поле будет пытаться разместиться в одной строке с соседним <code>inline</code> -полем. Ширина задается через <code>width</code> .
placeholder	string	Текст-подсказка внутри пустого поля.

4.3. Скрытие полей: два способа

Существует два метода, чтобы скрыть ненужные поля из интерфейса. Выбор зависит от контекста.

Способ 1: "hidden": true

Это основной способ. Вы добавляете `"hidden": true` прямо в конфигурацию поля.

```
"field_config": {  
  "project_id": {  
    "hidden": true  
  }  
}
```

Когда использовать: Когда поле нужно скрыть всегда и везде, где оно встречается.

Способ 2: "hidden_fields": [...]

Этот способ используется **только внутри universal_array_editor** . Вы указываете массив имен полей, которые нужно скрыть *только в форме редактирования элемента этого массива*.

```
"items": {  
  "widget": "universal_array_editor",  
  // ...  
  "hidden_fields": ["scene_number", "shot_number", "page_number"],  
  // ...  
}
```

Когда использовать: Когда одни и те же поля нужно в одном месте показать, а в другом (внутри списка) — скрыть. Это частый случай для ID и служебных полей.

4.4. Типы виджетов (widget)

entry

Однострочное текстовое поле. Идеально для коротких текстов, названий.

- width (integer): Ширина поля в символах.

```
"title": {  
  "widget": "entry",  
  "width": 50  
}
```

text_area

Многострочное текстовое поле. Для описаний, длинных текстов.

- width (integer): Ширина поля в символах.

- `height` (integer): Высота поля в строках.

```
"description": {  
  "widget": "text_area",  
  "height": 4  
}
```

combobox

Выпадающий список. Может быть как с редактированием, так и только для выбора.

- `values` (array): Массив строк со значениями для списка.
- `readonly` (boolean): Если `true`, пользователь сможет только выбирать из списка. Если `false` (по умолчанию), сможет также вводить свой вариант.

```
"genre": {  
  "widget": "combobox",  
  "values": ["сказка", "рассказ", "притча"],  
  "readonly": false  
}
```

spinbox

Поле для ввода чисел со стрелками "вверх/вниз".

- `width` (integer): Ширина поля.
- `from_` и `to` (integer): Минимальное и максимальное значения (берутся из JSON Schema, если не указаны).

```
"pages_min": {  
  "widget": "spinbox",  
  "width": 10  
}
```

checkbox

Флажок для булевых значений (`true` / `false`).

list_editor

Простой редактор для массива строк. Создает список полей ввода с кнопками "добавить/удалить".

- `add_button_text` (string): Текст на кнопке добавления нового элемента.

```
"main_characters": {
  "widget": "list_editor",
  "add_button_text": "+ Добавить персонажа"
}
```

nested_group

Виджет для редактирования вложенного JSON-объекта (не массива). Он рекурсивно создает форму для полей этого объекта.

```
"director_notes": {
  "hide_label": true,
  "widget": "nested_group"
}
```

5. universal_array_editor — Редактор массивов объектов

Это самый мощный и гибкий виджет, предназначенный для редактирования массива, где каждый элемент — это сложный объект. Например, список сцен в сценарии или список кадров.

5.1. Интерфейсы (interface)

Внешний вид редактора задается параметром `interface` .

interface	Описание
tabs	Каждый элемент массива отображается в отдельной вкладке. Удобно, когда элементов немного, но они сложные.

interface	Описание
accordion	Каждый элемент — это сворачиваемая панель (аккордеон).
list	Простой список с названиями элементов и кнопкой "Изменить" для открытия модального окна редактирования.
dropdown_selector	Рекомендуется для длинных списков. Элементы выбираются из выпадающего списка, а под ним появляется форма для редактирования выбранного элемента.

По умолчанию, если `interface` не указан, используется `dropdown_selector`.

Лучшая практика: Для `universal_array_editor` рекомендуется использовать ключ `interface`, а не `widget`. Ключ `widget` поддерживается для обратной совместимости, но `interface` семантически более корректен для описания внешнего вида редактора массивов.

5.2. Основные параметры

Параметр	Тип	Описание
interface	string	Тип интерфейса (см. таблицу выше).
add_button_text	string	Текст на кнопке добавления нового элемента.
item_name	string	"Имя" одного элемента. Используется для генерации текстов по умолчанию (например, "Новый <i>Кадр</i> ").
field_order	array	Очень важно! Массив строк с именами полей, определяющий порядок их отображения в форме редактирования <i>одного элемента массива</i> .
hidden_fields	array	Массив строк с именами полей, которые нужно скрыть из формы редактирования элемента.
field_groups	object	Позволяет создавать группы полей <i>внутри</i> формы редактирования одного элемента. Структура такая же, как у <code>field_groups</code> верхнего уровня. См. раздел 5.5.

5.3. Параметры для интерфейса tabs

Параметр	Тип	Описание
tab_title_field	string	Имя поля, значение которого будет использоваться в качестве заголовка для вкладки. Если не указано, будет "Элемент N".

Пример:

```
"pages": {
  "widget": "universal_array_editor",
  "interface": "tabs",
  "tab_title_field": "title" // Заголовком вкладки будет значение поля "title"
}
```

5.4. Параметры для интерфейса dropdown_selector

Параметр	Тип	Описание
display_field	string	Имя одного поля для показа в списке. Если задан display_fields , имеет меньший приоритет.
display_fields	array	Массив полей для составного отображения элемента (например, номер сцены + номер кадра).
display_separator	string	Разделитель между значениями полей из display_fields .
hide_selector_label	boolean	Скрыть метку над селектором. Текущая реализация не выводит метку; параметр зарезервирован (может не влиять на UI).
hide_edit_label	boolean	Скрыть заголовок блока редактирования выбранного элемента.

5.5. Конфигурация полей внутри элемента массива

Чтобы настроить виджеты для полей *внутри* объектов массива, вы добавляете их конфигурацию прямо в field_config для universal_array_editor , добавляя суффикс _field к имени поля.

Пример из shots.json :

В shots.json есть массив items . Каждый item (кадр) имеет поля shot_type , timing и т.д.

```
"shots": {
  "field_config": {
    "items": { // Конфигурация для всего массива
      "widget": "dropdown_selector",
      "display_fields": ["scene_number", "shot_number", "shot_type"],
      "display_separator": ".",
      "field_order": ["shot_type", "timing", "output_path", "english_prompt"], // Поряд

      // --- Конфигурация для полей ВНУТРИ кадра ---

      "shot_type_field": { // Обратите внимание на суффикс _field
        "widget": "combobox",
        "values": ["start", "end"],
        "inline": true, // Будет в одной строке с timing
        "width": 20
      },
      "timing_field": {
        "widget": "entry",
        "inline": true, // Будет в одной строке с shot_type
        "width": 20
      },
      "english_prompt_field": {
        "widget": "text_area",
        "height": 4
      }
    }
  }
}
```

5.6. Вложенная группировка полей (внутри universal_array_editor)

Вы можете группировать поля даже внутри формы редактирования одного элемента массива. Для этого используется ключ field_groups прямо внутри конфигурации universal_array_editor . Структура полностью аналогична field_groups верхнего уровня.

Пример из shots.json :

```

"items": {
  "widget": "dropdown_selector",
  "field_order": ["english_prompt", "video_prompt", "negative_prompt"], // Основные пол
  "field_groups": {
    "main_fields": {
      "title": "Основные промпты",
      "fields": ["english_prompt", "video_prompt", "negative_prompt"],
      "hide_label": false
    },
    "additional_fields": {
      "title": "Дополнительные настройки",
      "fields": ["*"], // Все остальные поля попадут в этот аккордеон
      "collapsible": true,
      "collapsed": true
    }
  }
}

```

Этот пример демонстрирует всю мощь системы:

1. Массив `items` отображается как выпадающий список.
2. В списке каждый элемент выглядит как `1.1.start`.
3. При выборе элемента открывается форма редактирования.
4. В этой форме поля идут в порядке, заданном `field_order`.
5. Поля `shot_type` и `timing` отображаются в одну строку (`inline: true`).
6. Поле `english_prompt` — это большое текстовое поле.
7. Внутри формы редактирования кадра поля сгруппированы: сначала идут основные промпты, а все остальные — спрятаны в сворачиваемый аккордеон "Дополнительные настройки".

Это полное руководство. Сохраните его как `doc/ui_config_guide.md` для будущих справок.

6. Дополнительные нюансы, поведение и ограничения

- **Заголовок для верхнеуровневых массивов:** если редактируемая схема — массив (например, `shots`), то единственным полем будет `items`. Заголовок группы можно задать через `field_groups` (как в конфиге) или через ключ `title` в корне схемы конфигурации (по умолчанию — "Элементы").

- **Когда используется items -конфиг:** если вы редактируете один объект элемента массива изолированно (данные — объект, а тип схемы — массив), генератор использует настройки из `field_config.items` для этого объекта (автоматически).
- **inline -поля:** поля с `inline: true`, идущие подряд, объединяются в одну строку. Как только встречается поле без `inline`, начинается новая строка. Ширина каждого виджета контролируется его `width` (если поддерживается виджетом).
- **Ограничения nested_group :** виджет вложенного объекта сейчас поддерживает `title` и конфигурацию дочерних полей, но не поддерживает сворачивание группы (`collapsible`) и скрытие заголовка группы (`hide_label`) на уровне самой группы.
- **field_order где работает:**
 - Поддерживается внутри `universal_array_editor` (упорядочивание полей формы элемента массива) и применяется через `items` -конфиг.
 - Для `nested_group` явная поддержка `field_order` отсутствует — порядок следует схеме/данным.
- **placeholder :** это не визуальный водяной знак в поле ввода. Он используется как начальное значение при создании новых элементов (через кнопку «Добавить») для строковых обязательных полей.
- **combobox :**
 - Если текущее значение не входит в `values`, оно будет добавлено в начало списка, чтобы не потерять существующие данные.
 - `readonly: false` (по умолчанию) позволяет вводить произвольный текст;
 - `readonly: true` — только выбор из списка.
- **list_editor :** при получении данных пустые строки отфильтровываются (возвращаются только непустые значения).
- **Переводы меток:** заголовок поля берётся из `field_config.label`, далее — из словаря `translations` (или `field_labels`), и только затем формируется автоматически из имени поля.
- **Валидация обязательности:** список `required` берётся из схемы (гибридной). Если явная JSON Schema не задана, как правило, список пустой, и проверяется только базовая корректность форматов.
- **collapsed и сворачивание:**
 - Работает для групп с `collapsible: true`.
 - Для несворачиваемых групп `collapsed` игнорируется.
- **Ключ interface vs widget :** для `universal_array_editor` предпочтительно указывать `interface` (`tabs`, `accordion`, `list`, `dropdown_selector`). `widget` поддерживается ради совместимости.

7. Примеры для каждого нюанса

7.1. dropdown_selector: display_field / display_fields / display_separator / hide_* метки

```
"items": {  
  "widget": "dropdown_selector",  
  "display_field": "title",           // простой режим: одно поле  
  "display_fields": ["scene_number", "shot_number", "shot_type"],  
  "display_separator": ".",          // сложный режим: составной заголовок  
  "hide_selector_label": true,       // метка над селектором скрыта  
  "hide_edit_label": true            // заголовок блока редактирования скрыт  
}
```

7.2. Верхнеуровневые массивы (items)

```
"shots": {  
  "field_groups": {  
    "shots_header": {  
      "title": "Управление кадрами", // заголовок для группы с массивом  
      "hide_label": true,  
      "fields": ["items"],           // верхнеуровневое поле массива  
      "layout": "vertical"  
    }  
  }  
}
```


7.3. Когда применяется конфиг items к объекту элемента массива

```
"shots": {
  "field_config": {
    "items": {
      "widget": "dropdown_selector",
      "field_order": ["shot_type", "timing"],
      "shot_type_field": {"widget": "combobox", "values": ["start", "end"]},
      "timing_field": {"widget": "entry", "width": 10}
    }
  }
}
```

Если в редактор передан ОДИН объект кадра (а тип схемы — массив), эти настройки из items будут использованы для формы этого объекта.

7.4. inline: группировка в одну строку подряд идущих inline-полей

```
"items": {
  "widget": "universal_array_editor",
  "field_order": ["scene_number", "shot_number", "shot_type", "timing", "output_path"],
  "scene_number_field": {"widget": "entry", "width": 8, "inline": true},
  "shot_number_field": {"widget": "entry", "width": 8, "inline": true},
  "shot_type_field": {"widget": "combobox", "values": ["start", "end"], "inline": true},
  "timing_field": {"widget": "entry", "width": 10, "inline": true},
  "output_path_field": {"widget": "entry", "width": 60} // начнёт НОВУЮ стр
}
```

7.5. Ограничения nested_group (нет collapsible/hide_label на уровне группы)

```
"director_notes": {
  "widget": "nested_group", // заголовок задаётся через title
  "title": "Режиссерские заметки"
  // Параметры collapsible/hide_label на уровне nested_group не поддерживаются
}
```

7.6. Где работает `field_order` (и где нет)

```
"items": {
  "widget": "universal_array_editor",
  "field_order": ["english_prompt", "video_prompt", "negative_prompt"] // работает здес
}
```

Для `nested_group` порядок полей задаётся структурой данных/схемой, `field_order` для него не применяется.

7.7. `placeholder` как дефолт при создании

```
"items": {
  "widget": "universal_array_editor",
  "english_prompt_field": {
    "widget": "text_area",
    "height": 3,
    "placeholder": "Опишите сцену на английском" // попадёт в новый элемент
  }
}
```

7.8. `combobox`: `readonly` и добавление текущего значения

```
"genre": {
  "widget": "combobox",
  "values": ["сказка", "рассказ", "притча"],
  "readonly": true // запрещает произвольный ввод
}
```

Если в данных встречается значение вне `values`, редактор добавит его в список, чтобы не потерять.

7.9. `list_editor`: фильтрация пустых строк

```
"main_characters": {
  "widget": "list_editor",
  "add_button_text": "+ Добавить персонажа"
}
```

При извлечении данных пустые элементы будут отброшены:

`["Лиса", "", "Медведь"] → ["Лиса", "Медведь"]` .

7.10. Приоритет меток (`label` → `translations` → `humanize`)

```
{
  "translations": {"camera_plan": "План камеры"},
  "screenplay": {
    "field_config": {
      "camera_plan": {
        "widget": "combobox",
        "label": "Схема кадра"                // приоритетнее translations
      }
    }
  }
}
```

Если `label` не задан, будет использовано `translations.camera_plan` . Иначе — `"Camera Plan"` → `"Camera Plan"` → `"Camera plan"` → `"Camera Plan"` (`humanize`).

7.11. Валидация `required` из гибридной схемы

```
// Если у поля есть ограничение в явной JSON Schema:
{
  "type": "object",
  "properties": {"title": {"type": "string", "minLength": 1}},
  "required": ["title"]
}
```

Редактор отметит ошибку, если `title` пустой/отсутствует.

7.12. `collapsed`: когда работает и когда игнорируется

```
"project_info": {
  "title": "Информация о проекте",
  "fields": ["concept", "director_notes"],
  "collapsible": true,
  "collapsed": true                // будет свёрнуто при старте
}
```

`collapsed` учитывается только если `collapsible: true`. Для обычных групп — игнорируется.

7.13. interface vs widget для universal_array_editor

// Рекомендуемый вариант

```
"screenplay": {  
  "field_config": {  
    "screenplay": {  
      "widget": "universal_array_editor",  
      "interface": "dropdown_selector"    // описывает внешний вид редактора  
    }  
  }  
}
```

// Устаревший (поддерживается для совместимости)

```
"items": {"widget": "dropdown_selector"}
```